Py-03 - Strutture Iterative (1)

Indice

1 Struttura iterativa `while` 2
- 1.1 Perché servono i cicli . . . 2
- 1.2 `while` - sintassi semplificata . . . 4
- 1.3 Schemi logici dei cicli `while` . . . 4
- 1.4 While - Numero di iterazioni noto in anticipo . . . 5
- 1.5 While - Numero di iterazioni non noto in anticipo . . . 6
- 1.6 Obiettivo del ciclo . . . 8

2 Esempi 9
- 2.1 Calcolo della media dei voti. Il numero di esami è noto. . . . 9
- 2.2 Calcolo della media pesata dei voti. Il numero di esami è noto. . . . 9
- 2.3 Calcolo spesa totale. Criterio di stop: inserisci 0 . . . 10
- 2.4 Calcolo spesa totale - solo importi interi. Criterio di stop: inserisci un carattere . . . 10
- 2.5 Inserimento interi. Criterio di stop: inserisci il carattere `x`. . . . 11
- 2.6 Scrivere un programma che legga 10 numeri e conti quanti sono positivi . . . 12
- 2.7 Scrivere un programma che legga dei voti e conti quanti voti sono sufficienti. Criterio di stop: inserisci un carattere . . . 12
- 2.8 Contare i voti sufficienti. Criterio di stop: al massimo 10 voti, uscita anticipata con un carattere . . . 13
- 2.9 Esercizi su rilevazione di evento . . . 14
- 2.10 Esercizi su accumulazione . . . 15
- 2.11 Esercizi su Ricerca di estremo . . . 15
- 2.12 Aggiornamento iterativo . . . 16

3 Sintassi completa: break, continue, else 17
- 3.1 `while` - sintassi completa . . . 17
- 3.2 Uscire dal ciclo con `break` . . . 17
- 3.3 Early stopping con `break` . . . 18
- 3.4 La clausola `else` . . . 18
- 3.5 `break` e cicli nidificati . . . 19
- 3.6 Saltare un'iterazione con `continue` . . . 23

4 Esercizi proposti (non in ordine di difficoltà) 24

1 Struttura iterativa `while`

1.1 Perché servono i cicli

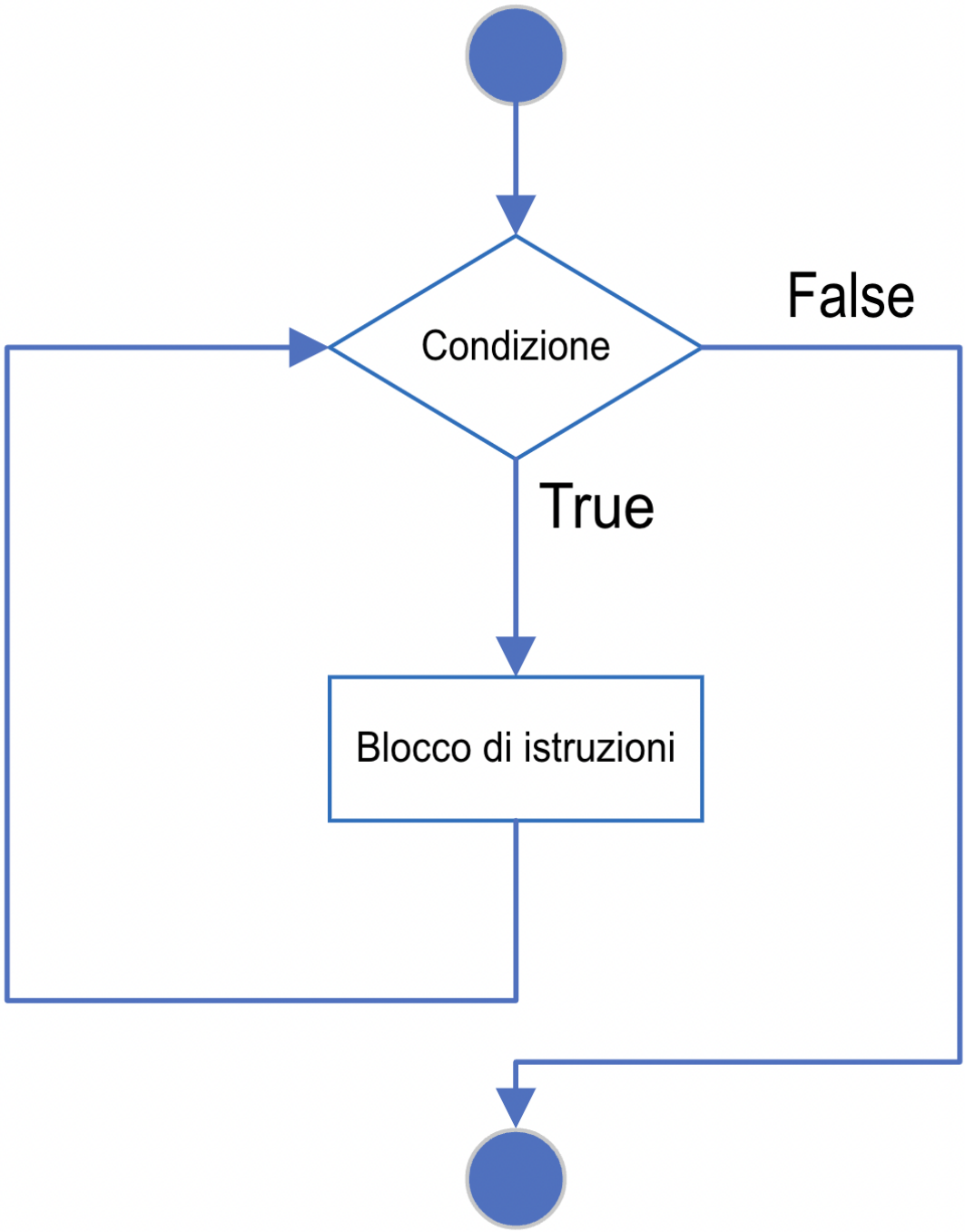
Spesso abbiamo necessità di ripetere un gruppo di istruzioni varie volte

```
# TABELLINE - stampa la tabellina del 3

n_tabellina=3
i = 1
risultato= n_tabellina * i
print( n_tabellina, '*', i, '=', risultato)
i = 2
risultato= n_tabellina * i
print( n_tabellina, '*', i, '=', risultato)
i = 3
risultato= n_tabellina * i
print( n_tabellina, '*', i, '=', risultato)
i = 4
risultato= n_tabellina * i
print( n_tabellina, '*', i, '=', risultato)
```

Se voglio stampare i primi 1000 valori questa soluzione è impraticabile...

Il costrutto `while` risolve il problema



- La `condizione` viene valutata
- Se la `condizione` è vera, viene eseguito il corpo del **while**, e la `condizione` viene rivalutata
- Se la `condizione` è falsa, si esce dal ciclo

1.2 `while` - sintassi semplificata

```
while condizione: # finché la condizione è vera...
    # corpo del while:
    # sequenza di istruzioni che vengono ripetute
    # finché condizione è vera
# NEXT INSTRUCTION
```

- La `condizione` viene valutata
- Se la `condizione` è vera, viene eseguito il corpo del while, e la `condizione` viene rivalutata
- Se la `condizione` è falsa, si esce dal ciclo (il controllo passa a `# NEXT INSTRUCTION`)

NOTA:

Se la condizione è inizialmente `False` il corpo del `while` non verrà mai eseguito.

Se la condizione è inizialmente `True` ed il corpo del `while` non la altera, si entra in un loop infinito. **È fondamentale dunque che il corpo del `while` abbia la possibilità di alterare la condizione**.

```
while True: # finché la condizione è vera...
    print('fermatemiiii!')
```

Per interrompere un programma bloccato in un ciclo infinito: premere **Ctrl+C** nel terminale, oppure usare il pulsante di stop (o di *interrupt* del kernel) dell'ambiente di sviluppo (Jupyter, VS Code, Spyder, …).

Soluzione esercizio Tabellina

```
# i = 1
# finché i è minore di 10 esegui le seguenti istruzioni:
#   calcola 3*i
#   stampa il risultato
#   incrementa i

i = 1
n_tabellina = 3
while i<=10:
    risultato =  n_tabellina *i
    print( n_tabellina, '*', i, '=', risultato)
    i = i+1
```

1.3 Schemi logici dei cicli `while`

Quando si utilizzano cicli per risolvere un problema è utile ragionare prima sulla **logica della ripetizione**, indipendentemente dal linguaggio di programmazione. Molti problemi che richiedono iterazione possono infatti essere ricondotti a pochi **schemi ricorrenti**. Riconoscere questi schemi aiuta a capire come organizzare il ragionamento prima ancora di scrivere il codice.

La logica del ciclo `while` è sempre la stessa: il corpo del ciclo viene eseguito finché la condizione rimane vera.

```
inizializzazione
while condizione:
    corpo del ciclo
    # durante il ciclo devono essere aggiornate
    # le variabili da cui dipende la condizione
```

Tuttavia è utile distinguere i diversi contesti logici in cui il ciclo viene utilizzato.

Criterio	Idea
Numero di iterazioni noto in anticipo	ripetere un'operazione un numero noto di volte
Numero di iterazioni non noto in anticipo	ripetere un'operazione finché una condizione, aggiornata a ogni iterazione, rimane `True`

- Nel primo schema il numero di iterazioni è noto o può essere determinato prima di iniziare il ciclo. Il ciclo viene quindi usato per eseguire un'operazione un certo numero di volte. Dal punto di vista logico è necessario inizializzare un contatore, verificare se il limite è stato raggiunto e aggiornare il contatore a ogni iterazione. Questo schema è tipico dei problemi in cui si deve elaborare un numero prefissato di valori.
- Nel secondo schema il numero di ripetizioni non è noto in anticipo. Il ciclo continua finché una certa condizione rimane vera e termina quando la condizione diventa falsa. Dal punto di vista logico l'attenzione non è sul numero di iterazioni ma sull'evoluzione dello stato del problema. A ogni iterazione alcune variabili vengono aggiornate e la condizione viene rivalutata.

1.4 While - Numero di iterazioni noto in anticipo

Utilizziamo un *contatore* che viene incrementato ad ogni iterazione ed utilizzato per valutare la condizione.

Esempio:

L'utente inserisce un numero che definisce il numero di iterazioni.

In seguito il programma chiede all'utente dei valori e li stampa

```
print('quanti numeri vuoi inserire?')
n = int(input())
i=0 # contatore delle iterazioni

while i < n:
    i = i + 1 # i viene incrementata
    # i += 1 # sintassi abbreviata
    valore = float(input())
    print('hai inserito il numero ', valore)
print('Istruzione fuori dal ciclo')
```

In questo esempio un contatore viene inizializzato a 0 e viene incrementato ad ogni esecuzione del corpo del `while`. La condizione `i<n` viene valutata all'inizio del ciclo (quando `i == 0`) e all'inizio di ogni iterazione.

Supponiamo che `n == 3`

Alla prima iterazione il valore di `i` passa da 0 a 1. Alla seconda iterazione il valore passa da 1 a 2. Alla terza iterazione il valore passa da 2 a 3. Al termine della terza iterazione la condizione `i < n` è `False` ed il ciclo si interrompe. Come previsto vengono eseguite `n == 3` iterazioni complete.

Errori comuni

```
n = int(input()) # dimenticare int()
i=0 # dimenticare di inizializzare i

while i<n:
    # sbagliare la condizione: i<=n, i>n, i != n

    i += 1 # dimenticare di incrementare
    # ...
```

1.5 While - Numero di iterazioni non noto in anticipo

Possiamo utilizzare il costrutto while per ripetere un blocco di istruzioni finché una certa condizione rimane vera. Il numero di iterazioni non è noto in anticipo, ma dipende da una condizione che viene valutata a ogni iterazione.

```
while condizione:
    # Blocco di istruzioni ripetute finché la condizione è vera
    # aggiornamento delle variabili da cui dipende la condizione
```

Il ciclo continuerà a eseguire il blocco di codice finché la condizione rimane `True`. Non appena la condizione diventa `False`, l'esecuzione del ciclo si interrompe.

Esempio: inserisci numeri; 0 per terminare

Qui la condizione di stop non è legata al numero di iterazioni svolto, ma al valore inserito in input.

```
valore = int(input())
while valore != 0 :
    print('hai inserito', valore)
    valore = int(input())
print('Istruzione fuori dal ciclo')
```

- Il primo valore viene letto prima del ciclo, perché la variabile `valore` è necessaria per valutare la condizione
- Se il primo valore è 0, la condizione è subito `False` e il ciclo non viene eseguito
- Quando durante il ciclo viene inserito 0, la condizione diventa `False` e il ciclo termina
- L'istruzione `print('hai inserito', valore)` non viene eseguita per il valore 0

Esempio: inserisci numeri; 0 per terminare

È possibile modificare il ciclo in modo che anche il primo input sia all'interno del ciclo. Poiché `valore` compare nella condizione del ciclo, la variabile deve avere un valore iniziale prima di

entrare nel ciclo. Possiamo quindi assegnare un valore iniziale che sia diverso da 0, in modo da forzare l'ingresso nel ciclo.

```
valore = 1
while valore != 0:
    valore = int(input())
    print('hai inserito', valore)
print('Istruzione fuori dal ciclo')
```

In questo caso l'istruzione

`print('hai inserito', valore)`

viene eseguita anche nell'ultima iterazione, quando `valore == 0`.

Nota: impostare un valore arbitrario purché diverso da 0 (`valore = 1`) per forzare l'ingresso nel ciclo potrebbe generare confusione. Spesso in questi casi si preferisce assegnare un valore diverso da 0 ma anche intrinsecamente diverso da tutti gli altri possibili valori di input. La prima inizializzazione può essere allora `valore = None`, dove `None` rappresenta l'assenza di valore (oggetto nullo).

Da notare che i cicli con prima lettura dentro o fuori dal ciclo sono strutturalmente diversi:

```
Schema A - prima lettura fuori dal ciclo
leggi valore
while valore != 0:
    elabora valore
    leggi valore


Schema B - prima lettura dentro il ciclo
inizializza valore
while valore != 0:
    leggi valore
    elabora valore
```

Variabile di controllo del ciclo

Possiamo utilizzare una variabile di controllo del ciclo.

```
continua = True # variabile di controllo
while continua: # cioè: continua == True
    valore = int(input())
    print('hai inserito', valore)
    continua = (valore!= 0)
```

L'espressione `valore != 0` produce un valore booleano (`True` o `False`). Questo valore viene assegnato alla variabile `continua`, che determina se il ciclo prosegue oppure termina.

Nota: la linea di codice

`continua = (valore != 0)`

equivale a

```
if valore == 0:
    continua = False
else:
    continua = True
```

1.6 Obiettivo del ciclo

Le categorie precedenti riguardano il **controllo del ciclo**. Gli schemi che seguono riguardano invece lo **scopo dell'elaborazione** svolta nel ciclo. Lo stesso schema può comparire sia con un numero di iterazioni noto sia con un numero non noto.

Obiettivo del ciclo	Idea centrale
Conteggio di occorrenze	contare quante volte si verifica un evento
Rilevazione di evento (flag)	verificare se un evento accade almeno una volta
Accumulazione	costruire progressivamente un risultato aggregato
Ricerca di estremo	trovare il massimo o il minimo tra i valori osservati
Aggiornamento iterativo	modificare progressivamente uno stato fino a una soglia

- **Conteggio di occorrenze**

Durante l'esecuzione del ciclo spesso è necessario **contare quante volte si verifica un certo evento**. Questo porta allo schema del **conteggio di occorrenze**. A ogni iterazione si verifica una condizione e, se questa è soddisfatta, si incrementa un contatore. Al termine del ciclo il contatore rappresenta il numero di volte in cui l'evento si è verificato. Questo schema è molto comune quando si analizzano sequenze di valori o si classificano dati.

- **Rilevazione di evento (flag)**

Un caso leggermente diverso è la **rilevazione di evento**. Qui non interessa sapere quante volte si verifica una certa condizione, ma soltanto se accade **almeno una volta**. In questo schema si utilizza una variabile logica che inizialmente indica che l'evento non è stato osservato. Se durante il ciclo la condizione si verifica, lo stato viene aggiornato. Alla fine del ciclo si può stabilire se l'evento è mai accaduto.

- **Accumulazione**

Un altro schema molto frequente è l'**accumulazione**. In questo caso il ciclo serve a costruire progressivamente un risultato combinando i valori elaborati a ogni iterazione. Si parte da un valore iniziale e lo si aggiorna ripetutamente secondo una certa regola. Esempi tipici sono la somma di valori, il calcolo di un totale o la costruzione di un risultato aggregato. Molti problemi più complessi, come il calcolo di una media, derivano direttamente da questo schema.

- **Ricerca di estremo**

Un'ulteriore situazione ricorrente è la **ricerca di un estremo**, come il valore massimo o minimo. Il ciclo confronta ogni nuovo valore con il miglior risultato trovato finora e aggiorna il risultato quando viene individuato un valore migliore. Dal punto di vista logico questo schema richiede di mantenere memoria della migliore soluzione trovata durante l'iterazione.

- **Aggiornamento iterativo**

Infine, in alcuni problemi il ciclo serve ad **aggiornare progressivamente uno stato** finché non si raggiunge una certa soglia o condizione di stabilità. In questo schema ogni iterazione applica una trasformazione allo stato corrente. Il ciclo termina quando la situazione soddisfa il criterio di arresto. Questo tipo di iterazione è tipico dei processi di simulazione o di avvicinamento graduale a un risultato.

2 Esempi

2.1 Calcolo della media dei voti. Il numero di esami è noto.

- Controllo del ciclo: numero di iterazioni noto in anticipo
- Obiettivo dell'elaborazione: accumulazione

```
print('Quanti esami hai sostenuto?')
n = int(input())
i = 0 # contatore
s = 0 # accumulatore
while i<n:
    print('inserisci il voto')
    v = int (input())
    s = s + v
    i += 1  # incremento del contatore- equivale a i = i + 1

print ('sono uscito dal ciclo')

media = s / n
print('media = ', media)
```

2.2 Calcolo della media pesata dei voti. Il numero di esami è noto.

- Controllo del ciclo: numero di iterazioni noto in anticipo
- Obiettivo dell'elaborazione: accumulazione

```
print('Quanti esami hai sostenuto?')
n = int(input())
i = 0
somma_voti_per_crediti = 0 # accumulatore
somma_crediti =0  #accumulatore

while i<n:
    print('inserisci il voto')
    v = int (input())
    print('inserisci i crediti')
    c = int (input())
    somma_crediti = somma_crediti + c
    somma_voti_per_crediti = somma_voti_per_crediti + v*c
```

```
    i += 1

print ('sono uscito dal ciclo')

media_pesata = somma_voti_per_crediti / somma_crediti
print('media pesata = ', media_pesata)
```

2.3 Calcolo spesa totale. Criterio di stop: inserisci 0

- Controllo del ciclo: numero di iterazioni non noto in anticipo
- Obiettivo dell'elaborazione: Accumulazione

```
s = 0 # accumulatore

print('inserisci il primo importo')
v = int(input())

while v !=0:
    s = s + v
    print('inserisci importo; 0 per uscire')
    v = int(input())

print ('sono uscito dal ciclo')

print('spesa totale = ', s)
```

2.4 Calcolo spesa totale - solo importi interi. Criterio di stop: inserisci un carattere

- Controllo del ciclo: numero di iterazioni non noto in anticipo
- Obiettivo dell'elaborazione: Accumulazione

```
s = 0
print('inserisci il primo importo(solo cifre), oppure una lettera per
 ↪  terminare')
v = input() # non converto ancora la stringa in intero

while v.isdigit():
    s = s + int(v)
    print('inserisci importo (solo cifre), oppure una lettera per
     ↪  terminare')
    v = input()

print ('sono uscito dal ciclo')

print('spesa totale = ', s)
```

Note:

- `v.isdigit()` rende `True` se la stringa `v` contiene solo cifre (quindi rappresenta un intero senza segno), per es. `"45"`; altrimenti rende `False`, per es. con `"-5"`, `"4.5"` o la stringa vuota `""`
- vedremo in seguito come gestire anche le stringhe che rappresentano `float`

Soluzione alternativa:

```
s = 0
continua = True

while continua:
    print('inserisci importo (solo cifre), oppure una lettera per
     ↪  terminare')
    v = input()
    if v.isdigit():
        s = s + int(v)
    else:
        continua = False

print ('sono uscito dal ciclo')
print('spesa totale = ', s)
```

2.5 Inserimento interi. Criterio di stop: inserisci il carattere `x`.

Altri caratteri diversi da `x` verranno ignorati. Se viene inserito un carattere diverso da `x` il programma deve chiedere un altro input e riesaminarlo.

- Controllo del ciclo: numero di iterazioni non noto in anticipo
- Obiettivo dell'elaborazione: adattabile a molti scopi
- soluzione ERRATA:

```
print("inserisci un numero intero oppure 'x' - altri caratteri non
 ↪  ammessi")
v = input()
if v !='x':
    if v.isdigit():
        v = int(v)
    else:
        print("hai inserito un carattere non ammesso")
        v = input()
        # a questo punto dovrei ricontrollare se v è digit oppure `x` in
         ↪  un ciclo potenzialmente infinito
```

- soluzione CORRETTA:

```
print("inserisci un numero intero oppure 'x' - altri caratteri non
 ↪  ammessi")
v = input()
while v != 'x':
    if v.isdigit():
        print(v)
    else:
        print("hai inserito un carattere non ammesso")
    v = input()
```

2.6 Scrivere un programma che legga 10 numeri e conti quanti sono positivi

- Controllo del ciclo: numero di iterazioni noto in anticipo
- Obiettivo dell'elaborazione: Conteggio di occorrenze

```
n = 10
i = 0
n_positivi = 0
while i<n:
    v = int (input())
    if v > 0:
        n_positivi += 1

    i += 1
print('hai inserito', n_positivi, 'numeri positivi')
```

Nota: In un'espressione matematica i valori `True` e `False` vengono interpretati come `1` e `0`, quindi è possibile sostituire la struttura condizionale con un costrutto più conciso.

```
#if v > 0:
#    n_positivi += 1

n_positivi += v>0 # True o False
```

2.7 Scrivere un programma che legga dei voti e conti quanti voti sono sufficienti. Criterio di stop: inserisci un carattere

- Controllo del ciclo: numero di iterazioni non noto in anticipo
- Obiettivo dell'elaborazione: Conteggio di occorrenze

```
voto_sufficienza = 6
# 6 per scuole superiori; 18 per univ

n_sufficienti = 0
```

```
print('inserisci voti; carattere per terminare')
v = input()
while v.isdigit(): # è convertibile in intero?
    voto = int(v)
    n_sufficienti += voto >= voto_sufficienza
    # equivale a
    # if voto >= voto_sufficienza:
    #    n_sufficienti += 1

    v = input()

print('hai inserito ',n_sufficienti, 'voti sufficienti')
```

2.8 Contare i voti sufficienti. Criterio di stop: al massimo 10 voti, uscita anticipata con un carattere

Criterio di stop: al massimo 10 voti, ma il programma termina prima se l'utente inserisce un carattere.

- Controllo del ciclo: numero di iterazioni massimo noto in anticipo, ma è presente un criterio di uscita anticipata
- Obiettivo dell'elaborazione: Conteggio di occorrenze

```
n = 10
voto_sufficienza = 6

n_voti = 0
n_sufficienti = 0

print('Inserisci fino a', n, 'voti; un carattere non numerico per
 ↪  terminare')
v = input()

while v.isdigit() and n_voti < n:
    voto = int(v)
    if voto >= voto_sufficienza:
        n_sufficienti += 1
    n_voti += 1
    if n_voti < n:
        v = input()

print('Voti inseriti:', n_voti)
print('Voti sufficienti:', n_sufficienti)
```

Si noti che se l'utente inserisce 10 valori numerici, dopo l'ultimo inserimento `n_voti` vale `n` e, grazie all'ultimo `if`, non viene richiesto un valore oltre il decimo. La soluzione è poco elegante, perché ripete il controllo `n_voti < n` sia nella condizione sia nel corpo del `while`. Vedremo fra poco il costrutto `break`, che permette di formulare altre soluzioni.

Ecco una soluzione più elegante che fa uso di una variabile booleana.

```
n = 10
voto_sufficienza = 6

n_voti = 0
n_sufficienti = 0
continua = True

print('Inserisci fino a', n, 'voti; un carattere non numerico per
 ↪  terminare')

while continua:
    v = input()
    if v.isdigit():
        voto = int(v)
        if voto >= voto_sufficienza:
            n_sufficienti += 1
        n_voti += 1
        continua = n_voti < n
    else:
        continua = False

print('Voti inseriti:', n_voti)
print('Voti sufficienti:', n_sufficienti)
```

La lettura avviene all'inizio di ogni iterazione, quindi non serve più l'`input()` prima del ciclo. Ci sono due modi di uscire. Se l'utente inserisce un carattere non numerico, il ramo `else` mette `continua = False`. Se invece ha appena inserito il decimo voto, `continua = n_voti < n` diventa `False`. In entrambi i casi il `while` termina senza chiedere altri valori.

2.9 Esercizi su rilevazione di evento

Tutti gli esercizi su conteggio di occorrenze si possono trasformare in rilevazione di evento. Si usa una variabile `flag` al posto del contatore.

Per esempio: su 10 voti, verificare se almeno un voto è maggiore di 27.

```
n = 10
i = 0
soglia = 27
flag = False

print('Inserisci voti')

while i < n:
    voto = int(input())

    if voto > soglia:
        flag = True

    i += 1
```

```
if flag:
    print ('almeno un voto è maggiore di ',soglia )
else:
    print ('nessun voto è maggiore di ',soglia )
```

2.10 Esercizi su accumulazione

- da svolgere con numero di iterazioni noto o non noto in anticipo, oppure con un numero di iterazioni massimo ma con un criterio di stop anticipato.

Esempi:

Calcolo della somma totale, della media aritmetica, pesata, armonica, della varianza (la mediana invece richiede di memorizzare e ordinare tutti i valori: servono strumenti che vedremo più avanti, come le liste). Sono esercizi analoghi a quelli visti nelle sezioni in cui abbiamo analizzato i due diversi criteri di stop.

2.11 Esercizi su Ricerca di estremo

- da svolgere con numero di iterazioni noto o non noto in anticipo, oppure con un numero di iterazioni massimo ma con un criterio di stop anticipato.

Esempio: su 10 voti, trovare il voto massimo.

La variabile `voto_max` memorizza **il valore massimo osservato fino a quel momento**.

All'inizio `voto_max` viene inizializzato a `0`. A ogni iterazione il programma legge un nuovo voto e lo confronta con il massimo corrente. Se il nuovo voto è maggiore, `voto_max` viene aggiornato. In questo modo, durante l'esecuzione del ciclo, `voto_max` conserva sempre **il voto più alto incontrato finora**. Al termine delle `n` iterazioni, la variabile contiene **il massimo tra tutti i voti inseriti**.

```
i = 0
n = 10

voto_max = 0
while i<n:
    voto = int (input())
    if  voto > voto_max:
        voto_max = voto

    i +=1
print('il voto massimo è ...', voto_max)
```

Stesso esercizio, ma esaminando 10 valori di temperatura. In questo caso non possiamo inizializzare a `0` la variabile `temp_max`. Cosa succederebbe con temperature tutte negative?

Soluzione:

- Si legge la prima temperatura prima del ciclo e la si usa per inizializzare `temp_max`.
- Successivamente, a ogni iterazione, la nuova temperatura viene confrontata con il massimo corrente. Se è maggiore, `temp_max` viene aggiornato. In questo modo la variabile conserva sempre la temperatura più alta osservata finora.

```
i = 0
n = 10

# prima lettura
temperatura = float(input())
temp_max = temperatura

i += 1

while i < n:
    temperatura = float(input())

    if temperatura > temp_max:
        temp_max = temperatura

    i += 1

print('la temperatura massima è ...', temp_max)
```

2.12 Aggiornamento iterativo

Esempi:

Data una popolazione ed un tasso di crescita, dopo quanto tempo la popolazione raddoppierà?
Data una popolazione ed un tasso di decrescita, dopo quanto tempo la popolazione dimezzerà?
Dopo quanto tempo scenderà al di sotto di una soglia data?

Data una cifra iniziale ed un tasso di interesse, quanti anni devono passare per raddoppiare la cifra?

```
# data una cifra iniziale ed un tasso di interesse,
# quanti anni devono passare per raddoppiare la cifra?

interesse = float(input('inserisci il tasso di interesse (es. 0.05 per
 ↪  5%): '))
capitale_iniziale = int(input('inserisci capitale iniziale'))
obiettivo = 2 * capitale_iniziale

capitale_attuale = capitale_iniziale
anno = 0
while capitale_attuale < obiettivo:
    anno += 1
    capitale_attuale = capitale_attuale * (1 + interesse)
    print ('anno', anno, 'ora hai', capitale_attuale)
    #    print ("anno ",anno,f"ora hai {capitale_attuale:.2f}") # stampa
     ↪  due decimali
```

```
print ('fine del ciclo: ora hai', capitale_attuale)
print ('anni trascorsi', anno)
```

3 Sintassi completa: break, continue, else

3.1 `while` - sintassi completa

Nel ciclo `while` possiamo usare due parole chiave speciali per controllare l'esecuzione.

- `break`: interrompe completamente il ciclo e passa all'istruzione successiva fuori dal `while`. Si noti che in caso di cicli annidati (*nested*), cioè inseriti uno dentro l'altro, il `break` interrompe solo il ciclo più interno.
- `continue`: salta l'iterazione corrente e passa all'iterazione successiva, senza eseguire il resto del codice all'interno del ciclo ma senza interrompere il ciclo

Inoltre il `while` è provvisto anche di una clausola `else`

```
while condizione:
    # Blocco di codice eseguito finché la condizione è vera
else:
    # Blocco eseguito solo se il ciclo termina normalmente (senza break)
```

- Il blocco dentro `while` viene eseguito finché `condizione == True`.
- Se il ciclo termina normalmente (perché `condizione == False`), viene eseguito il blocco `else`.
- Se il ciclo viene interrotto con `break`, il blocco `else` non viene eseguito.

https://docs.python.org/it/3.13/reference/compound_stmts.html#while

3.2 Uscire dal ciclo con `break`

Esempio: chiedere all'utente di inserire un numero finché non digita 0:

- Soluzione senza `break`

```
numero = int(input())
while numero != 0:
    print("Hai inserito:", numero)
    numero = int(input())
print("Hai scelto di uscire dal ciclo.")
```

- Soluzione con `break`

```
while True:  # Un ciclo teoricamente infinito
    print("Inserisci un numero (0 per uscire): ")
    numero = int(input())
    if numero == 0:
        print("Hai scelto di uscire dal ciclo.")
        break  # Esce dal ciclo
    print("Hai inserito:", numero)
```

3.3 Early stopping con `break`

- Itera 10 volte, oppure esci prima se l'utente inserisce 0

Soluzione senza `break`

```
i = 0
n = 10
v = int (input())
print(v)
i += 1
while i<n  and v!=0:
    v = int (input())
    print(v)
    i += 1
print('sono fuori dal ciclo')
if v ==0:
    print('sono uscito perché hai inserito 0')
```

Nota: a differenza della soluzione con `break`, questa versione stampa anche lo `0` prima di uscire dal ciclo.

Soluzione con `break`

```
i = 0
n = 10
while i<n:
    v = int (input())
    if v == 0:
        print('è stato inserito 0')
        break
    print(v)
    i += 1
print('sono fuori dal ciclo')
```

3.4 La clausola `else`

Se il ciclo `while` termina normalmente (cioè perché `condizione == False`) viene eseguito il blocco `else`. In questo modo è facile definire due comportamenti diversi a seconda della condizione che ha portato a terminare il ciclo

```
i = 0
n = 10
while i<n:
    v = int (input())
    if v == 0:
        print('è stato inserito 0 - esco dal ciclo')
        break
    print(v)
    i += 1
else:
    print ('Ciclo terminato perché sono state eseguite tutte le iterazioni
     ↪ richieste')

print('sono fuori dal ciclo')
```

3.5 `break` e cicli nidificati

```
# ricavo gli indici degli elementi di una matrice quadrata 10 * 10
n = 10
i = 0
while i<n:
    j = 0
    i += 1
    while j<n:
        j += 1
        print("elemento", i, j)
```

```
elemento 1 1
elemento 1 2
elemento 1 3
elemento 1 4
elemento 1 5
elemento 1 6
elemento 1 7
elemento 1 8
elemento 1 9
elemento 1 10
elemento 2 1
elemento 2 2
elemento 2 3
elemento 2 4
elemento 2 5
elemento 2 6
elemento 2 7
elemento 2 8
elemento 2 9
elemento 2 10
elemento 3 1
elemento 3 2
elemento 3 3
elemento 3 4
```

```
elemento 3 5
elemento 3 6
elemento 3 7
elemento 3 8
elemento 3 9
elemento 3 10
elemento 4 1
elemento 4 2
elemento 4 3
elemento 4 4
elemento 4 5
elemento 4 6
elemento 4 7
elemento 4 8
elemento 4 9
elemento 4 10
elemento 5 1
elemento 5 2
elemento 5 3
elemento 5 4
elemento 5 5
elemento 5 6
elemento 5 7
elemento 5 8
elemento 5 9
elemento 5 10
elemento 6 1
elemento 6 2
elemento 6 3
elemento 6 4
elemento 6 5
elemento 6 6
elemento 6 7
elemento 6 8
elemento 6 9
elemento 6 10
elemento 7 1
elemento 7 2
elemento 7 3
elemento 7 4
elemento 7 5
elemento 7 6
elemento 7 7
elemento 7 8
elemento 7 9
elemento 7 10
elemento 8 1
elemento 8 2
elemento 8 3
elemento 8 4
elemento 8 5
elemento 8 6
elemento 8 7
elemento 8 8
elemento 8 9
elemento 8 10
elemento 9 1
elemento 9 2
elemento 9 3
```

```
elemento 9 4
elemento 9 5
elemento 9 6
elemento 9 7
elemento 9 8
elemento 9 9
elemento 9 10
elemento 10 1
elemento 10 2
elemento 10 3
elemento 10 4
elemento 10 5
elemento 10 6
elemento 10 7
elemento 10 8
elemento 10 9
elemento 10 10
```

- Il `break` termina l'esecuzione del ciclo in cui è inserito (in questo caso il ciclo **esterno**)

```
# ricavo gli indici degli elementi di una matrice quadrata 10 * 10
n = 10
i = 0
while i<n:
    j = 0
    i += 1
    if i==2:
        break
    while j<n:
        j += 1
        print("elemento", i, j)
```

```
elemento 1 1
elemento 1 2
elemento 1 3
elemento 1 4
elemento 1 5
elemento 1 6
elemento 1 7
elemento 1 8
elemento 1 9
elemento 1 10
```

- Il `break` termina l'esecuzione del ciclo in cui è inserito (in questo caso il ciclo **interno**)

```
# ricavo gli indici degli elementi di una matrice quadrata 10 * 10
n = 10
i = 0
while i<n:
    j = 0
    i += 1

    while j<n:
        j += 1
```

```
        if i==2:
            break
        print("elemento", i, j)
```

```
elemento 1 1
elemento 1 2
elemento 1 3
elemento 1 4
elemento 1 5
elemento 1 6
elemento 1 7
elemento 1 8
elemento 1 9
elemento 1 10
elemento 3 1
elemento 3 2
elemento 3 3
elemento 3 4
elemento 3 5
elemento 3 6
elemento 3 7
elemento 3 8
elemento 3 9
elemento 3 10
elemento 4 1
elemento 4 2
elemento 4 3
elemento 4 4
elemento 4 5
elemento 4 6
elemento 4 7
elemento 4 8
elemento 4 9
elemento 4 10
elemento 5 1
elemento 5 2
elemento 5 3
elemento 5 4
elemento 5 5
elemento 5 6
elemento 5 7
elemento 5 8
elemento 5 9
elemento 5 10
elemento 6 1
elemento 6 2
elemento 6 3
elemento 6 4
elemento 6 5
elemento 6 6
elemento 6 7
elemento 6 8
elemento 6 9
elemento 6 10
elemento 7 1
elemento 7 2
elemento 7 3
elemento 7 4
```

```
elemento 7 5
elemento 7 6
elemento 7 7
elemento 7 8
elemento 7 9
elemento 7 10
elemento 8 1
elemento 8 2
elemento 8 3
elemento 8 4
elemento 8 5
elemento 8 6
elemento 8 7
elemento 8 8
elemento 8 9
elemento 8 10
elemento 9 1
elemento 9 2
elemento 9 3
elemento 9 4
elemento 9 5
elemento 9 6
elemento 9 7
elemento 9 8
elemento 9 9
elemento 9 10
elemento 10 1
elemento 10 2
elemento 10 3
elemento 10 4
elemento 10 5
elemento 10 6
elemento 10 7
elemento 10 8
elemento 10 9
elemento 10 10
```

3.6 Saltare un'iterazione con `continue`

La parola chiave `continue` permette di saltare l'iterazione corrente e passare alla successiva. `continue` non interrompe il `while`.

Esempio: stampare i numeri da 1 a 10, ma saltare i multipli di 3

```python
numero = 0

while numero < 10:
    numero += 1
    if numero % 3 == 0:
        # se il resto della divisione tra numero e 3 è ==0
        continue  # Salta il resto del codice e passa alla prossima
        ↪ iterazione
    print(numero)
```

4 Esercizi proposti (non in ordine di difficoltà)

Provate a risolvere gli esercizi proposti in vari modi, se possibile:

- Definendo un numero massimo di iterazioni.
- Definendo un criterio di stop.
- Usando entrambi i criteri.
- Usando (o non usando) `break` e `continue`.
- Usando operatori booleani per combinare due condizioni.
- dato un numero intero, calcolare il fattoriale
- calcolo della media escludendo il voto più basso
- calcolo della media escludendo il voto più basso e il voto più alto (media robusta.)
- calcolare i primi n elementi della successione di Fibonacci https://it.wikipedia.org/wiki/Successione_di_Fibonacci $F_n = F_{n-1} + F_{n-2}$
- calcolo della spesa finale, applicando uno sconto a seconda di determinate condizioni (per esempio, sconto legato al numero di acquisti o all'importo totale)
- sommare solo i valori che soddisfano una condizione
- Scrivere un programma che chiede all'utente un valore `n` e stampa i numeri PARI fino ad `n`. Se `n==10` il programma deve stampare `2, 4, 6, 8, 10`
- Scrivere un programma che acquisisce una serie di frasi e stampa la frase più lunga. Hint: usare `len(stringa)`
- Scrivere un programma che calcola il numero di individui in una popolazione noto il tasso di crescita. Ogni iterazione rappresenta un anno. Il programma termina quando la popolazione è raddoppiata o dimezzata (il tasso di crescita potrebbe essere negativo) rispetto al valore iniziale. Variante: l'utente aggiorna il tasso di crescita ogni anno.
- Scrivere un programma che chieda all'utente un insieme di numeri e li elabori nel seguente modo. Inizialmente il programma chiede all'utente il numero di valori (addendi) da inserire. L'inserimento termina quando l'utente inserisce il carattere `x`, oppure quando viene raggiunto il numero di addendi richiesto. Se l'utente inserisce il valore `-1`, la somma corrente viene azzerata e `-1` non viene conteggiato tra gli addendi (quindi non riduce il numero di valori ancora da inserire).
- Dato un numero intero positivo, contare il numero di cifre. NB è richiesto di manipolare il numero, non la sua rappresentazione mediante stringa!

```
# Soluzione
print("Inserisci un numero intero, calcolerò il numero di cifre")
n= int(input())
i=0
while n>=1:
    i= i+1
    n = n // 10
print('Numero di cifre ---> ',i)
```

Nota: con input `0` il programma risponde 0 cifre; per gestire questo caso serve un controllo in più.

- Dati due numeri interi positivi, trovare il loro massimo comune divisore (MCD) usando l'Algoritmo di Euclide https://it.wikipedia.org/wiki/Algoritmo_di_Euclide
- Scrivere un programma che dica se p è un numero primo. Varianti: se p è un numero primo, dire se è un numero primo di Sophie Germain[1]. Dire se è un numero primo di Mersenne[2]
- Scrivere un programma che chiede all'utente dei numeri e li somma. Il programma termina quando la somma è maggiore di 100. Se dopo 10 iterazioni la somma non è ancora maggiore di 100, il programma termina lo stesso.

```
print('Inserisci i dati, termino quando la somma eccede 100 o dopo 10
 ↪  inserimenti!')
s=0
i=0
while s<=100 and i<10:
    # DE MORGAN
    # not(s>100 or i>=10)
    v = int(input())
    s = s+v
    i += 1
print('somma:', s)
print('numero inserimenti:', i)

# VARIANTE:
# stampare un messaggio che spieghi se il ciclo è terminato
# perché somma >100 o perché si sono superate le 10 iterazioni
```

- Conversione base 2-> base 10. Scrivere un programma che chieda all'utente il numero di bit che intende utilizzare, e poi chieda il valore dei bit a partire dal *msb* (most significant bit, il bit più significativo). Il programma deve stampare il valore in base 10.

```
# SOLUZIONE:
print("Preparati ad inserire un numero intero espresso in base 2, a
 ↪  partire da MSB")
print("Quanti bit?")

n= int(input())
print("Ok, mi preparo a convertire un numero di ", n, " bit. Comincia
 ↪  l'inserimento")
i=0
v=0
while i<n:
    v= v*2
    b= int(input())
    v= v+b
    i+=1
```

[1] https://it.wikipedia.org/wiki/Numero_primo_di_Sophie_Germain
[2] https://it.wikipedia.org/wiki/Numero_primo_di_Mersenne

```
print('In base 10: ---> ',v)
```

- Dato un numero intero positivo, stampare il numero al contrario. NB è richiesto di manipolare il numero, **non** la sua rappresentazione mediante stringa!

```
# Soluzione
n_reverse=0
print('Inserisci un numero')
n= int(input())
while n>=1:
    resto= n%10
    n= n // 10
    n_reverse = n_reverse*10
    n_reverse = n_reverse + resto
print(n_reverse)
```

Nota: gli zeri finali si perdono (`1230` diventa `321`), perché come numero `0321` vale `321`.

- Scrivere un programma che contiene un numero segreto e chiede all'utente di indovinarlo. Variante: Il computer sceglie un numero segreto tra 1 e 100, e l'utente ha solo 5 tentativi per indovinarlo. Dopo ogni tentativo il programma deve dire "Troppo alto" o "Troppo basso" finché l'utente indovina o esaurisce i tentativi. Suggerimento: usa la funzione `random.randint()` per generare un numero casuale.

```
import random

# genera un numero (pseudo) casuale tra 1 e 100
numero_segreto = random.randint(1,100)

n= 5
i = 0
while i<n:
    i = i+1
    print('indovina il numero')
    v = int(input())
    if v == numero_segreto:
        print('INDOVINATO!')
        break
    elif v<numero_segreto: # riferito a IF
        print('Hai inserito un numero troppo piccolo')
    else:
        print('Hai inserito un numero troppo grande')
else: # riferito a WHILE
    print('mi dispiace, hai esaurito i tentativi')
    print('il numero era ', numero_segreto)


# La maggior parte dei linguaggi di programmazione hanno la struttura
 ↪ WHILE
# senza la clausola ELSE
```

```
# Formulate una soluzione alternativa SENZA la clausola ELSE del WHILE
# in modo che si possa implementare in altri linguaggi di programmazione
```